



MÓDULO 8

DESARROLLO SEGURO Y GESTIÓN DE IDENTIDADES

TAREA 1

BEATRIZ RODRÍGUEZ GUTIÉRREZ
MÁSTER EN CIBERSEGURIDAD
TAREA 1- MÓDULO 8

ÍNDICE

Introducción	1
Fundamentos de TOTP (Time-based One-Time Password)	2
TOTP (Time-based One-Time Password)	2
Explicación del Programa	4
Lenguaje y Librerías Utilizadas	4
Diseño del Software	4
Estructura del Proyecto	7
Guía de Uso y Ejecución	8
Instalación de Dependencias	8
Ejecución del Sistema Completo	8
Análisis sobre Seguridad y Propuestas de Mejora	11
CÓDIGOS	12

Introducción

La tarea presentada en este documento describe la tarea1 del módulo 8, ésta consiste en la implementación de un software que permita generar un código QR para añadir un servicio en una aplicación de gestión de códigos temporales (como Google Authenticator), así como validar un código TOTP introducido por el usuario para verificar si es correcto o no.

Para dar respuesta a dicho enunciado, se ha desarrollado en Python un sistema de escritorio compuesto por dos aplicaciones:

- **admin.py (El panel de gestión):** Es el programa que utiliza el administrador para registrar a los nuevos usuarios. Su función principal es generar un código QR personalizado para cada usuario. Cuando el usuario escanea este QR con su teléfono móvil, su cuenta queda vinculada.
- **cliente.py (La puerta de acceso):** Es la pantalla que utiliza el usuario final para intentar iniciar sesión. Aquí debe introducir el código temporal (TOTP) que le muestra su móvil. Si el sistema comprueba que el código es válido, le permite acceder a su área privada.

A lo largo del documento se describe el funcionamiento del protocolo RFC 6238 en el que se basa la solución, se explican las decisiones técnicas adoptadas durante el desarrollo y se detalla el procedimiento necesario para ejecutar y probar el sistema.

Fundamentos de TOTP (Time-based One-Time Password)

TOTP (Time-based One-Time Password)

El algoritmo TOTP es una extensión del algoritmo de contraseñas de un solo uso (OTP), concretamente del algoritmo HOTP (HMAC-based One-Time Password) definido previamente en el RFC 4226. Mientras que el algoritmo original HOTP se basa en un factor de movimiento consistente en un "contador de eventos" que incrementa matemáticamente, TOTP sustituye ese contador por un "valor de tiempo". El objetivo de basar el algoritmo en el tiempo es proporcionar contraseñas de muy corta duración (short-lived OTP values), lo cual es altamente deseable para mejorar la seguridad del sistema.

La expresión matemática del algoritmo (Según RFC 6238, Sección 4.2)

El documento oficial define el cálculo de la contraseña temporal con la siguiente expresión base:

$$\text{TOTP} = \text{HOTP}(K, T)$$

En esta fórmula:

- **K:** representa el secreto compartido (shared secret) entre el usuario y el servidor.
- **T:** es un número entero que representa el número de "pasos de tiempo" (time steps) que hay entre un tiempo inicial determinado (T0) y el tiempo Unix actual.

Para calcular exactamente el valor de T, el protocolo exige aplicar la siguiente fórmula:

$$T = (\text{Current Unix time} - T0) / X$$

- **Current Unix time:** Es la hora actual del sistema.
- **T0:** Es el tiempo Unix en el que se empiezan a contar los pasos de tiempo (su valor por defecto es 0, es decir, el Unix epoch).
- **X:** Es el parámetro del sistema que define el paso de tiempo en segundos (su valor por defecto son 30 segundos)

En esta división, el estándar especifica que se debe aplicar la función matemática floor (que redondea el resultado eliminando los decimales). De este modo, y tomando como ejemplo un T0 = 0 y un paso X = 30, el valor matemático de T será 1 cuando el tiempo transcurrido sea de 59 segundos. Tan solo un segundo después, al llegar a los 60 segundos, el valor de T cambiará a 2.

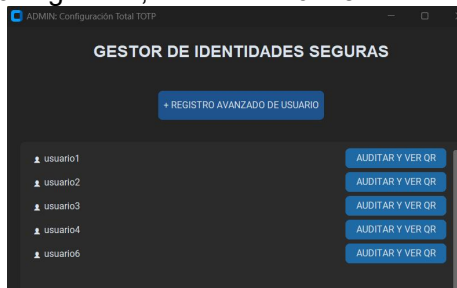
Parámetros configurables del Protocolo

El RFC 6238 especifica un conjunto de parámetros o variables del sistema que influyen en cómo se generan y validan estas contraseñas:

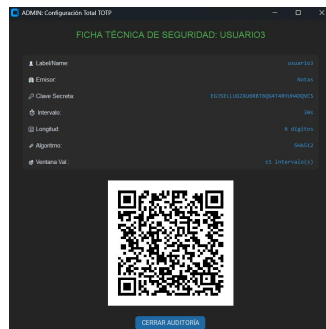
Parámetro	Descripción
Paso de tiempo (Time-Step Size - X)	Define, en segundos, la duración de cada ventana de validez de la contraseña. El RFC recomienda un valor por defecto de 30 segundos, considerándolo el equilibrio ideal entre seguridad y usabilidad. Si se configura un paso de tiempo muy grande, se aumenta la ventana de exposición para que un atacante intercepte y use la clave; pero si es muy corto, el usuario podría no tener tiempo de introducirlo.
Tiempo inicial (T0)	Es el tiempo Unix exacto desde el cual se empiezan a contar los pasos de tiempo. El documento establece su valor por defecto en 0 (conocido como Unix epoch).
Secreto compartido (Key - K)	Es la clave única de cada usuario. El estándar indica que este secreto debe ser generado de forma completamente aleatoria y recomienda que su longitud coincida con la longitud de salida de la función HMAC que se vaya a utilizar.

Función criptográfica (HMAC)	Aunque el algoritmo base original utilizaba la función HMAC-SHA-1, el RFC 6238 actualiza el estándar permitiendo explícitamente que las implementaciones de TOTP utilicen funciones hash más seguras, como HMAC-SHA-256 o HMAC-SHA-512.
Ventana de transmisión (Transmission delay window)	Debido a la latencia de las redes, el código puede llegar al servidor segundos después de haber sido generado. El RFC indica que el servidor debe configurar una política para aceptar cierta demora, recomendando que como máximo se acepte un paso de tiempo de retraso por culpa de la red.
Límite de resincronización (Clock drift)	Los relojes del servidor y del móvil del usuario pueden desincronizarse con el tiempo. El protocolo recomienda configurar el sistema de validación con un límite de pasos de tiempo "hacia adelante" y "hacia atrás" que está dispuesto a tolerar. Por ejemplo, configurar la aceptación de hasta dos ventanas de tiempo anteriores (unos 60 segundos de margen) para evitar que un usuario con el reloj ligeramente desajustado sea rechazado.

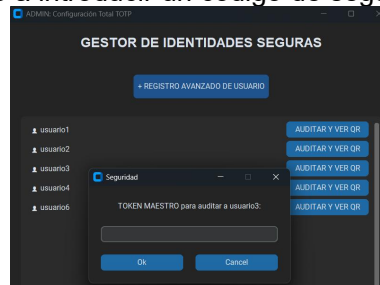
Registrar usuarios y crear "llaves": Permite al administrador crear cuentas nuevas y configurar reglas de seguridad personalizadas para cada una (por ejemplo, decidir si la contraseña durará 30 o 60 segundos, o si tendrá 6 u 8 números).



Generar los códigos QR: Una vez configurado el usuario, el programa dibuja en la pantalla un código QR. El usuario escanea este QR con su aplicación móvil (como Google Authenticator) y, desde ese momento, su teléfono queda sincronizado con el sistema.



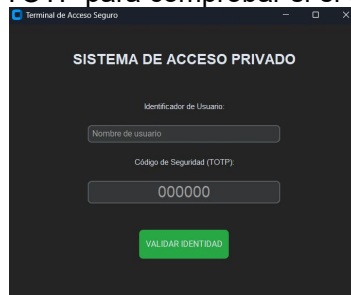
Protegerse a sí mismo: Este panel es tan crítico que tiene su propio cerrojo llamado "Token Maestro". Si el administrador quiere ver la información secreta o el QR de alguien, el programa le obliga primero a introducir un código de seguridad de su propio móvil.



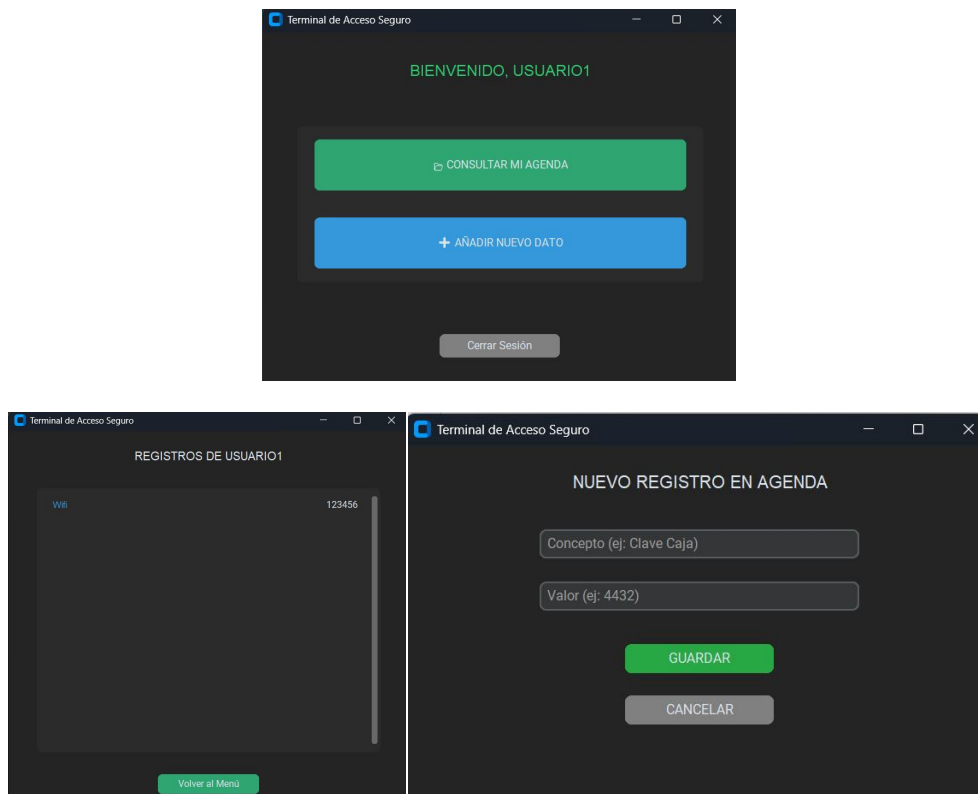
2. cliente.py (El Terminal de Acceso del Usuario)

Este programa es la "puerta de entrada" o inicio de sesión. Lo que hace es:

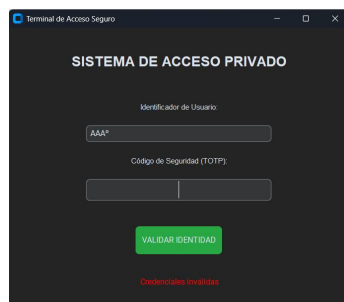
Validar el acceso: El usuario final abre este programa e introduce su nombre y el número de 6 dígitos que le está mostrando su móvil en ese segundo. El programa aplica las matemáticas del estándar TOTP para comprobar si el código es correcto.



Entregar la información: Si el código es válido, la puerta se abre y el programa le permite al usuario acceder y gestionar una agenda personal cifrada con sus datos privados.



Confundir a los atacantes (Validación Ciega): Como medida de seguridad, si alguien intenta hackear la entrada poniendo un usuario falso o un código inventado, el programa siempre responde con el mismo mensaje genérico: "Credenciales inválidas". Así, un atacante no puede adivinar si ha fallado porque el usuario no existe, o porque el código estaba mal.



En resumen, el programa `admin.py` crea y reparte las llaves de seguridad en forma de códigos QR, mientras que el programa `cliente.py` actúa como el vigilante de la puerta que verifica los códigos temporales para dejar pasar a los usuarios a su agenda privada.

Persistencia de Datos

Para el almacenamiento continuo de la información, el sistema utiliza archivos en formato JSON (`cuentas_seguras.json` y `agendas_usuarios.json`).

Cabe destacar que esta decisión de diseño se ha tomado con fines estrictamente académicos y didácticos, con el objetivo de facilitar el desarrollo y la evaluación de la aplicación.

Desde una perspectiva de seguridad para un entorno de producción real, esta persistencia de datos no es la más óptima por los siguientes motivos:

- **Vulnerabilidad de los datos en reposo (Texto plano):** En la implementación actual, los archivos JSON se almacenan en el disco duro en texto plano. Esto implica que si un atacante lograra acceso físico o lógico al sistema de archivos del servidor, podría leer directamente las claves criptográficas compartidas (secrets base32) de todos los usuarios. Con esta información, el atacante podría clonar las cuentas en su propio dispositivo móvil y generar códigos TOTP válidos de forma fraudulenta.
- **Exposición de identidades:** Los nombres o identificadores de los usuarios se utilizan como claves maestras de los diccionarios dentro del JSON, también en texto claro. Cualquier acceso no autorizado a este archivo revelaría de manera inmediata la lista completa de usuarios registrados en el sistema, facilitando posibles ataques dirigidos.

A pesar de la debilidad del formato de texto plano, el programa sí aplica correctamente el principio de seguridad de "Separación de responsabilidades" al dividir la información en dos archivos independientes:

- **cuentas_seguras.json:** Generado y gestionado exclusivamente por admin.py, contiene únicamente los parámetros técnicos de seguridad y los secretos.
- **agendas_usuarios.json:** Generado y gestionado exclusivamente por cliente.py, almacena los datos de la agenda personal del usuario.

Esta separación física es una buena práctica de diseño porque garantiza que un compromiso o corrupción de los datos privados de la agenda no exponga ni dañe la configuración criptográfica del sistema, y viceversa.

En un despliegue profesional, la estructura de los archivos se mantendría, pero su contenido debería protegerse aplicando técnicas criptográficas para los datos en reposo y aplicando funciones hash a los identificadores de usuario.

Estructura del Proyecto

Para reforzar el principio de separación de responsabilidades (Separation of Concerns) y mantener un entorno de desarrollo organizado, el código fuente y los datos del sistema se distribuyen en la siguiente estructura de directorios:

```

proyecto/
├── admin/
│   ├── admin.py
│   └── data/           ← cuentas_seguras.json y QR
└── cliente/
    ├── cliente.py
    └── data/           ← agendas_usuarios.json
  
```

Esta distribución modular divide el sistema en dos bloques completamente aislados:

Directorio admin/ (Gestión y Seguridad): Contiene el núcleo de la administración.

- El script admin.py actúa como panel de control protegido por el token maestro.
- La carpeta data/ dentro de este directorio tiene un propósito crítico: almacenar el archivo cuentas_seguras.json (que contiene los secretos Base32 y parámetros técnicos de todos los usuarios) y guardar temporalmente las imágenes de los códigos QR generados durante el aprovisionamiento.

Directorio cliente/ (Acceso y Usuario): Contiene el portal del usuario final.

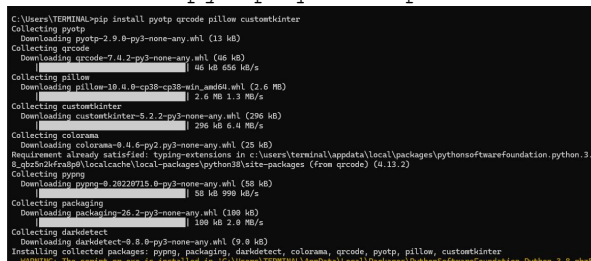
- El script cliente.py implementa el terminal de validación ciega para verificar los códigos TOTP.
- Su respectiva carpeta data/ aísla el archivo agendas_usuarios.json, el cual contiene únicamente los datos de negocio o información privada de los usuarios.

Guía de Uso y Ejecución

Instalación de Dependencias

Antes de iniciar el sistema, es un requisito disponer de Python 3.8 o una versión superior instalada en el equipo. La instalación de todas las dependencias externas requeridas se realiza mediante el gestor de paquetes pip ejecutando el siguiente comando:

```
pip install customtkinter pyotp qrcode pillow
```



```
C:\Users\TERMINAL>pip install customtkinter pyotp qrcode pillow
Collecting pyotp
  Downloading pyotp-2.9.0-py3-none-any.whl (13 kB)
Collecting qrcode
  Downloading qrcode-7.4.2-py3-none-any.whl (48 kB)
Collecting pillow
  Downloading pillow-10.0.0-cp38-cp38-win_amd64.whl (2.6 MB)
Collecting customtkinter
  Downloading customtkinter-5.2.2-py3-none-any.whl (296 kB)
Collecting colorama
  Downloading colorama-0.4.6-py2.py3-none-any.whl (25 kB)
Requirement already satisfied: typing_extensions in c:\users\terminal\appdata\local\packages\pythonsoftwarefoundation.python.3.8.china2fr6ph\localcache\local-packages\python38\site-packages (from qrcode) (4.11.2)
Collecting pyper
  Downloading pyper-0.20220715.0-py3-none-any.whl (59 kB)
Collecting packaging
  Downloading packaging-20.2-py3-none-any.whl (309 kB)
Collecting dashdetect
  Downloading dashdetect-0.8.0-py3-none-any.whl (9.9 kB)
Installing collected packages: pyper, packaging, dashdetect, colorama, qrcode, pyotp, pillow, customtkinter
WARNING: The script pyper is installed to 'C:\Users\TERMINAL\AppData\Local\ Packages\PythonSoftwareFoundation.Python.3.8.china2fr6ph\Scripts' but it is not a script.
```

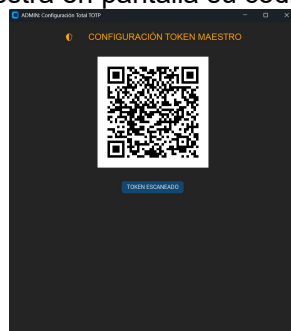
Cabe destacar que el módulo hashlib forma parte de la biblioteca estándar de Python, por lo que no requiere ninguna instalación adicional.

Ejecución del Sistema Completo

El sistema ha sido diseñado para ejecutarse siguiendo un orden lógico. A continuación, se describen los pasos desde la configuración inicial hasta el acceso de un usuario:

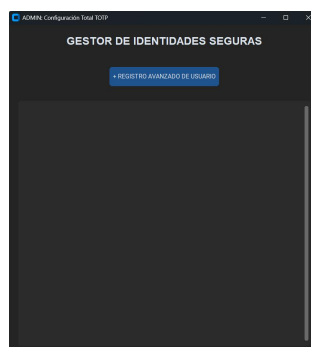
Paso 1: Ejecución inicial de admin.py

Se arranca el panel de control ejecutando admin.py. Como es la primera vez y no existe el archivo cuentas_seguras.json, el sistema detecta que falta la clave admin_master. Automáticamente, invoca la función crear_token_maestro(), la cual genera el secreto maestro del administrador y muestra en pantalla su código QR correspondiente.



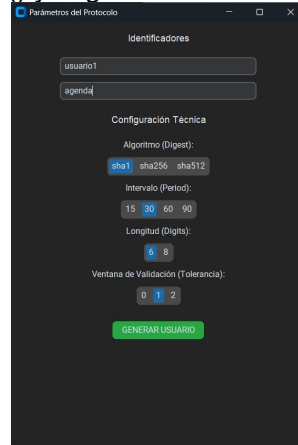
Paso 2: Escaneo del Token Maestro

El administrador debe escanear el QR mostrado utilizando Google Authenticator u otra aplicación como FreeOTP+. Esta operación se realiza una única vez; por seguridad, este QR inicial no volverá a mostrarse. Tras escanearlo, el administrador pulsa el botón 'TOKEN ESCANEADO' y el sistema carga el menú principal invocando mostrar_menu_principal().



Paso 3: Registro de un Nuevo Usuario

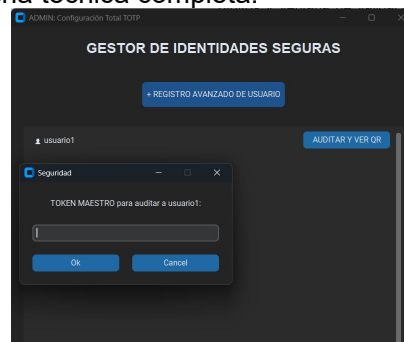
Desde el menú principal, el administrador selecciona la opción '+ REGISTRO AVANZADO DE USUARIO', lo que abre la ventana `ventana_registro_avanzado()`. En este formulario se configuran de forma personalizada los parámetros técnicos exigidos por el RFC 6238 (etiqueta, emisor, algoritmo hash, intervalo de tiempo, longitud de dígitos y ventana de validación). Al confirmar, el sistema genera un secreto aleatorio mediante `pyotp.random_base32()` y lo guarda en el archivo JSON.



The screenshot shows a window titled 'Parámetros del Protocolo'. It has two main sections: 'Identificadores' and 'Configuración Técnica'. Under 'Identificadores', there are input fields for 'usuario1' (containing 'usuario1') and 'agencia1' (containing 'agencia1'). Under 'Configuración Técnica', there are several settings: 'Algoritmo (Digest)' with radio buttons for 'sha1', 'sha256', and 'sha512' (where 'sha1' is selected); 'Intervalo (Period)' with a dropdown menu showing '15', '30', '60', and '90' (where '30' is selected); 'Longitud (Digits)' with a dropdown menu showing '6' and '8' (where '6' is selected); and 'Ventana de Validación (Tolerancia)' with a dropdown menu showing '0', '1', and '2' (where '1' is selected). At the bottom, there is a green button labeled 'GENERAR USUARIO'.

Paso 4: Auditoría y Entrega del QR al Usuario

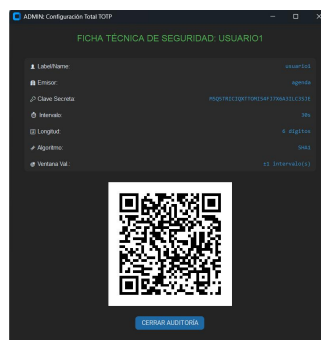
Para entregar el acceso, el administrador selecciona el perfil del usuario recién creado y pulsa 'AUDITAR Y VER QR'. El sistema, como medida de seguridad, solicita introducir el código TOTP maestro mediante la función `verificar_maestro()`. Tras introducirlo correctamente, la función `mostrar_detalle_finales()` genera el código QR personalizado del usuario y muestra su ficha técnica completa.



The screenshot shows a window titled 'ADMIN: Configuración Total TOTP' with a sub-header 'GESTOR DE IDENTIDADES SEGURAS'. It features a button '+ REGISTRO AVANZADO DE USUARIO' and a list of users. The first user, 'usuario1', is selected, and a button 'AUDITAR Y VER QR' is visible. A modal window titled 'Seguridad' is open, asking for the 'TOKEN MAESTRO para auditar a usuario1:'. It has an input field and 'Ok' and 'Cancel' buttons.

Paso 5: Aprovisionamiento en Google Authenticator u otra aplicación.

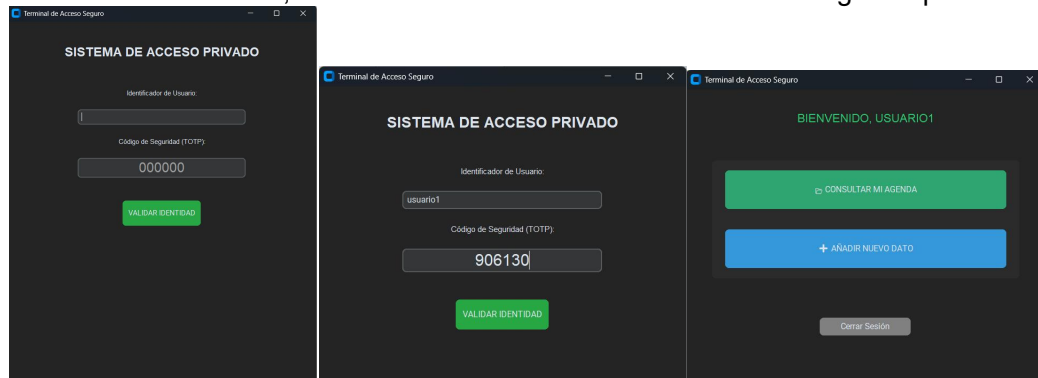
El usuario final recibe ese código QR y lo escanea con su aplicación móvil. Desde ese instante, su teléfono comienza a generar los códigos TOTP temporales, sincronizados de forma exacta con los parámetros que configuró el administrador. Tras esto, pulsamos CERRAR.



The screenshot shows a window titled 'ADMIN: Configuración Total TOTP' with a sub-header 'FICHA TÉCNICA DE SEGURIDAD: USUARIO1'. It displays a list of technical parameters on the left and their corresponding values on the right. The parameters are: 'Identificador', 'Emisor', 'Clase Secreto', 'Intervalo', 'Longitud', 'Algoritmo', and 'Ventana Val.'. The values are: 'usuario1', 'agencia1', 'PROTECTOR TOTP SECRET KEY', '30', '6', 'sha1', and '1'. A large QR code is displayed in the center. At the bottom, there is a blue button labeled 'CERRAR AUDITORIA'.

Paso 6: Acceso mediante cliente.py

Finalmente, el usuario ejecuta el portal de acceso con `python cliente.py`. En la pantalla de inicio de sesión, introduce su identificador y el código TOTP numérico que le muestra su móvil en ese momento. El método `validar()` procesa las credenciales y, si la verificación es exitosa, concede al usuario acceso al menú de su agenda personal.



NOTA sobre la TOLERANCIA

Si la tolerancia (el parámetro técnico conocido como window o ventana de validación) se configura con el valor 0, significa que el sistema solo aceptará como válido el código TOTP generado en ese instante exacto, sin admitir intervalos de tiempo adyacentes.

En la práctica, configurar la tolerancia a 0 provoca lo siguiente:

- **Cero margen para desajustes de reloj (Clock drift):** Es muy común que el reloj interno del móvil del usuario y el reloj del servidor tengan un ligero desfase de algunos segundos. Al poner el valor a 0, el sistema no compensará esta desincronización y rechazará el código.
- **Cero margen para latencia de red:** Si el usuario se retrasa un poco al teclear el número o si la conexión a internet es lenta, el código podría llegar al servidor justo cuando el intervalo de 30 segundos acaba de cambiar. Al estar en 0, el sistema no aceptará el código del "intervalo anterior" y denegará el acceso.

En resumen, poner la tolerancia a 0 ofrece el nivel más alto y estricto de seguridad porque la ventana para usar el código es la mínima posible, pero afecta negativamente a la usabilidad.

Análisis sobre Seguridad y Propuestas de Mejora

Si bien el sistema cumple holgadamente con los objetivos académicos planteados, se han identificado tres mejoras para elevar el nivel de protección para cumplir con estándares exigidos en entornos de producción:

Cifrado AES-256 para los archivos JSON en reposo

En la implementación actual, los archivos locales se almacenan en texto plano, por lo que un atacante con acceso físico al sistema de archivos podría leer directamente los secretos TOTP. Se propone cifrar ambos archivos utilizando el estándar AES-256-GCM (modo autenticado), derivando la clave de cifrado mediante PBKDF2-HMAC-SHA256 a partir de una contraseña maestra. La librería `cryptography.fernet` de Python podría proporcionar esta capa de seguridad de forma eficiente.

Límite de intentos de login (Rate Limiting)

Actualmente, el sistema no restringe el número de intentos de autenticación fallidos. Aunque el espacio de un código TOTP de 6 dígitos es de un millón de combinaciones (10⁶) y su validez de tiempo es corta, la falta de limitación facilita ataques de fuerza bruta automatizados. La mejora consistiría en implementar un patrón de exponencial backoff (bloqueos temporales progresivos de 1s, 5s, 30s, 300s) tras cada fallo consecutivo.

Hashing de identificadores de usuario

Los nombres de usuario se almacenan actualmente en texto claro como claves del diccionario dentro del archivo JSON. Cualquier acceso no autorizado a este archivo revelaría de inmediato la lista completa de usuarios registrados. Se propone como medida sustituir estas claves en texto plano por su hash SHA-256.

CÓDIGOS

A continuación se muestran los códigos empleados para el desarrollo del proyecto.

```
admin.py
import customtkinter as ctk
import pyotp
import qrcode
import json
import os
from PIL import Image

ctk.set_appearance_mode("dark")

# --- Rutas ---
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
DATA_DIR = os.path.join(BASE_DIR, "data")
os.makedirs(DATA_DIR, exist_ok=True)

DB_FILE = os.path.join(DATA_DIR, "cuentas_seguras.json") # solo admin escribe aquí
QR_DIR = DATA_DIR # QR guardados en admin/data/
# ---

class AdminPanel(ctk.CTk):
    def __init__(self):
        super().__init__()
        self.title("ADMIN: Configuración Total TOTP")
        self.geometry("650x900")
        self.imagen_qr_actual = None
        self.cuentas = self.cargar_datos()

        if "admin_master" not in self.cuentas:
            self.crear_token_maestro()
        else:
            self.mostrar_menu_principal()

    def cargar_datos(self):
        default = {"usuarios": {}}
        if os.path.exists(DB_FILE):
            try:
                with open(DB_FILE, "r") as f:
                    return json.load(f)
            except:
                return default
        return default

    def guardar_datos(self):
        with open(DB_FILE, "w") as f:
            json.dump(self.cuentas, f, indent=4)

    def crear_token_maestro(self):
        self.limpiar()
        ctk.CTkLabel(self, text=" CONFIGURACIÓN TOKEN MAESTRO", font=("bold", 20),
text_color="orange").pack(pady=20)
        secret = pyotp.random_base32()
        self.cuentas["admin_master"] = {"secret": secret, "interval": 30, "digits": 6, "algo":
"sha1"}
        self.guardar_datos()
        uri = pyotp.totp.TOTP(secret).provisioning_uri(name="ADMIN", issuer_name="MasterKey")
        qr_path = os.path.join(QR_DIR, "admin_master.png")
        qrcode.make(uri).save(qr_path)
        pil_img = Image.open(qr_path)
        self.imagen_qr_actual = ctk.CTkImage(light_image=pil_img, dark_image=pil_img,
size=(250, 250))
        ctk.CTkLabel(self, image=self.imagen_qr_actual, text="").pack(pady=10)
        ctk.CTkButton(self, text="TOKEN ESCANEADO",
command=self.mostrar_menu_principal).pack(pady=20)

    def mostrar_menu_principal(self):
        self.limpiar()
        ctk.CTkLabel(self, text="GESTOR DE IDENTIDADES SEGURAS", font=("Helvetica", 22,
"bold")).pack(pady=20)
        ctk.CTkButton(self, text="+ REGISTRO AVANZADO DE USUARIO", fg_color="#1f538d",
height=40,
command=self.ventana_registro_avanzado).pack(pady=20)
        self.scroll = ctk.CTkScrollableFrame(self, height=500)
        self.scroll.pack(fill="both", padx=20, pady=10)
        self.actualizar_lista()
```

```

def ventana_registro_avanzado(self):
    v = ctk.CTkToplevel(self)
    v.title("Parámetros del Protocolo")
    v.geometry("450x650")
    v.attributes("-topmost", True)

    ctk.CTkLabel(v, text="Identificadores", font=("bold", 14)).pack(pady=10)
    e_name = ctk.CTkEntry(v, placeholder_text="Nombre/Email (Label)", width=300)
    e_name.pack(pady=5)
    e_issuer = ctk.CTkEntry(v, placeholder_text="Emisor (App Name)", width=300)
    e_issuer.insert(0, "CiberApp_Pro")
    e_issuer.pack(pady=5)

    ctk.CTkLabel(v, text="Configuración Técnica", font=("bold", 14)).pack(pady=10)

    ctk.CTkLabel(v, text="Algoritmo (Digest):").pack()
    s_algo = ctk.CTkSegmentedButton(v, values=["sha1", "sha256", "sha512"])
    s_algo.set("sha1")
    s_algo.pack(pady=5)

    ctk.CTkLabel(v, text="Intervalo (Period):").pack()
    s_time = ctk.CTkSegmentedButton(v, values=["15", "30", "60", "90"])
    s_time.set("30")
    s_time.pack(pady=5)

    ctk.CTkLabel(v, text="Longitud (Digits):").pack()
    s_digits = ctk.CTkSegmentedButton(v, values=["6", "8"])
    s_digits.set("6")
    s_digits.pack(pady=5)

    ctk.CTkLabel(v, text="Ventana de Validación (Tolerancia):").pack()
    s_win = ctk.CTkSegmentedButton(v, values=["0", "1", "2"])
    s_win.set("1")
    s_win.pack(pady=5)

    def guardar():
        nombre = e_name.get()
        if not nombre:
            return
        self.cuentas["usuarios"][nombre] = {
            "secret": pyotp.random_base32(),
            "interval": int(s_time.get()),
            "digits": int(s_digits.get()),
            "algo": s_algo.get(),
            "window": int(s_win.get()),
            "issuer": e_issuer.get()
        }
        self.guardar_datos()
        v.destroy()
        self.actualizar_lista()

    ctk.CTkButton(v, text="GENERAR USUARIO", fg_color="#28a745",
command=guardar).pack(pady=20)

    def actualizar_lista(self):
        for w in self.scroll.winfo_children():
            w.destroy()
        for u in self.cuentas["usuarios"]:
            f = ctk.CTkFrame(self.scroll)
            f.pack(fill="x", pady=2)
            ctk.CTkLabel(f, text=f" {u}").pack(side="left", padx=10)
            ctk.CTkButton(f, text="AUDITAR Y VER QR", width=140,
command=lambda x=u: self.verificar_maestro(x)).pack(side="right",
padx=5)

    def verificar_maestro(self, usuario):
        token = ctk.CTkInputDialog(text=f"TOKEN MAESTRO para auditar a {usuario}:",
title="Seguridad").get_input()
        if token:
            admin_s = self.cuentas["admin_master"]["secret"]
            if pyotp.totp.TOTP(admin_s).verify(token.strip()):
                self.mostrar_detalle finales(usuario)

    def mostrar_detalle finales(self, usuario):
        self.limpiar()
        u = self.cuentas["usuarios"][usuario]

        ctk.CTkLabel(self, text=f"FICHA TÉCNICA DE SEGURIDAD: {usuario.upper()}",
font=("bold", 18), text_color="#4CAF50").pack(pady=15)

```



```

detalles_frame = ctk.CTkFrame(self)
detalles_frame.pack(pady=10, padx=30, fill="x")

config_info = [
    (" Label/Name:", usuario),
    (" Emisor:", u["issuer"]),
    (" Clave Secreta:", u["secret"]),
    (" Intervalo:", f"{u['interval']}s"),
    (" Longitud:", f"{u['digits']} digitos"),
    (" Algoritmo:", u["algo"].upper()),
    (" Ventana Val.:", f"#{u['window']} intervalo(s)")
]

for i, val in config_info:
    row = ctk.CTkFrame(detalles_frame, fg_color="transparent")
    row.pack(fill="x", padx=10, pady=2)
    ctk.CTkLabel(row, text=i, font=("bold", 12)).pack(side="left")
    ctk.CTkLabel(row, text=val, font=("Consolas", 11),
text_color="#3498db").pack(side="right")

import hashlib
digest_mod = (hashlib.sha1 if u["algo"] == "sha1"
               else hashlib.sha256 if u["algo"] == "sha256"
               else hashlib.sha512)

uri = pyotp.totp.TOTP(
    u["secret"], interval=u["interval"], digits=u["digits"], digest=digest_mod
).provisioning_uri(name=usuario, issuer_name=u["issuer"])

qr_path = os.path.join(QR_DIR, f"qr_{usuario}.png")
qrcode.make(uri).save(qr_path)
pil_img = Image.open(qr_path)
self.imagen_qr_actual = ctk.CTkImage(light_image=pil_img, dark_image=pil_img,
size=(250, 250))

ctk.CTkLabel(self, image=self.imagen_qr_actual, text="").pack(pady=10)
ctk.CTkButton(self, text="CERRAR", command=self.mostrar_menu_principal).pack(pady=10)

def limpiar(self):
    for w in self.wininfo_children():
        w.destroy()

if __name__ == "__main__":
    AdminPanel().mainloop()

```

cliente.py

```

import customtkinter as ctk
import pyotp
import json
import os
import hashlib
from datetime import datetime

ctk.set_appearance_mode("dark")
ctk.set_default_color_theme("green")

# --- Rutas ---
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
DATA_DIR = os.path.join(BASE_DIR, "data")
os.makedirs(DATA_DIR, exist_ok=True)

# cuentas_seguras.json lo escribe admin; apuntamos a admin/data/ desde cliente/
ADMIN_DATA_DIR = os.path.join(BASE_DIR, "..", "admin", "data")
DB_CONFIG = os.path.join(ADMIN_DATA_DIR, "cuentas_seguras.json")

DB_AGENDAS = os.path.join(DATA_DIR, "agendas_usuarios.json") # solo cliente escribe aquí
# ---

class AppCliente(ctk.CTk):
    def __init__(self):
        super().__init__()
        self.title("Terminal de Acceso Seguro")
        self.geometry("550x650")
        self.db_agendas = self.cargar_agendas()
        self.mostrar_login()

```

```

def cargar_config_seguridad(self):
    if os.path.exists(DB_CONFIG):
        try:
            with open(DB_CONFIG, "r") as f:
                return json.load(f)
        except:
            return None
    return None

def cargar_agendas(self):
    if os.path.exists(DB_AGENDAS):
        try:
            with open(DB_AGENDAS, "r") as f:
                return json.load(f)
        except:
            return {}
    return {}

def guardar_agendas(self):
    with open(DB_AGENDAS, "w") as f:
        json.dump(self.db_agendas, f, indent=4)

def mostrar_login(self):
    self.limpiar_interfaz()

    ctk.CTkLabel(self, text="SISTEMA DE ACCESO PRIVADO", font=("Helvetica", 22,
"bold")).pack(pady=40)

    ctk.CTkLabel(self, text="Identificador de Usuario:", font=("bold", 12)).pack(pady=5)
    self.entry_user = ctk.CTkEntry(self, placeholder_text="Nombre de usuario", width=300)
    self.entry_user.pack(pady=10)

    ctk.CTkLabel(self, text="Código de Seguridad (TOTP):", font=("bold", 12)).pack(pady=5)
    self.entry_otp = ctk.CTkEntry(self, placeholder_text="000000", justify="center",
font=("Arial", 24), width=300)
    self.entry_otp.pack(pady=10)

    ctk.CTkButton(self, text="VALIDAR IDENTIDAD", fg_color="#28a745",
height=45, command=self.validar).pack(pady=30)

    self.lbl_info = ctk.CTkLabel(self, text="", text_color="red")
    self.lbl_info.pack()

def validar(self):
    data = self.cargar_config_seguridad()
    if not data:
        self.lbl_info.configure(text="Error: Sistema no configurado")
        return

    user_input = self.entry_user.get().strip().lower()
    otp_input = self.entry_otp.get().strip()

    acceso_concedido = False

    if user_input in data["usuarios"]:
        u_cfg = data["usuarios"][user_input]
        algos = {"sha1": hashlib.sha1, "sha256": hashlib.sha256, "sha512": hashlib.sha512}
        totp = pyotp.totp.TOTP(
            u_cfg["secret"],
            interval=u_cfg["interval"],
            digits=u_cfg["digits"],
            digest=algos.get(u_cfg.get("algo", "sha1"))
        )
        if totp.verify(otp_input, valid_window=u_cfg.get("window", 0)):
            acceso_concedido = True

    if acceso_concedido:
        print(f"[LOG {datetime.now()}] Acceso exitoso para: {user_input}")
        self.mostrar_menu_post_login(user_input)
    else:
        print(f"[LOG {datetime.now()}] Intento fallido para: {user_input}")
        self.lbl_info.configure(text="Credenciales inválidas")
        self.entry_otp.delete(0, "end")

def mostrar_menu_post_login(self, usuario):
    self.limpiar_interfaz()
    ctk.CTkLabel(self, text=f"BIENVENIDO, {usuario.upper()}",
font=("bold", 18), text_color="#2ecc71").pack(pady=30)

    frame_btms = ctk.CTkFrame(self)

```

```

        frame_btns.pack(pady=20, padx=40, fill="both")

        ctk.CTkButton(frame_btns, text="CONSULTAR MI AGENDA", height=60,
                      command=lambda: self.mostrar_agenda(usuario)).pack(pady=15, padx=20,
                      fill="x")
        ctk.CTkButton(frame_btns, text="+ AÑADIR NUEVO DATO", height=60, fg_color="#3498db",
                      command=lambda: self.ventana_añadir_datos(usuario)).pack(pady=15,
                      padx=20, fill="x")

        ctk.CTkButton(self, text="Cerrar Sesión", fg_color="gray",
                      command=self.mostrar_login).pack(pady=40)

    def mostrar_agenda(self, usuario):
        self.limpiar_interfaz()
        ctk.CTkLabel(self, text=f"REGISTROS DE {usuario.upper()}", font=("bold",
        16)).pack(pady=20)

        scroll = ctk.CTkScrollableFrame(self, width=450, height=350)
        scroll.pack(pady=10)

        datos = self.db_agendas.get(usuario.lower(), [])
        if not datos:
            ctk.CTkLabel(scroll, text="No hay datos registrados.").pack(pady=20)

        for t, d in datos:
            f = ctk.CTkFrame(scroll)
            f.pack(fill="x", pady=2, padx=5)
            ctk.CTkLabel(f, text=t, font=("bold", 12), text_color="#3498db").pack(side="left",
            padx=10)
            ctk.CTkLabel(f, text=d).pack(side="right", padx=10)

        ctk.CTkButton(self, text="Volver al Menú",
                      command=lambda: self.mostrar_menu_post_login(usuario)).pack(pady=20)

    def ventana_añadir_datos(self, usuario):
        self.limpiar_interfaz()
        ctk.CTkLabel(self, text="NUEVO REGISTRO EN AGENDA", font=("bold", 16)).pack(pady=20)

        e_titulo = ctk.CTkEntry(self, placeholder_text="Concepto (ej: Clave Caja)", width=300)
        e_titulo.pack(pady=10)
        e_valor = ctk.CTkEntry(self, placeholder_text="Valor (ej: 4432)", width=300)
        e_valor.pack(pady=10)

        def guardar():
            t, v = e_titulo.get(), e_valor.get()
            if t and v:
                u_key = usuario.lower()
                if u_key not in self.db_agendas:
                    self.db_agendas[u_key] = []
                self.db_agendas[u_key].append([t, v])
                self.guardar_agendas()
                self.mostrar_menu_post_login(usuario)

        ctk.CTkButton(self, text="GUARDAR", fg_color="#28a745", command=guardar).pack(pady=20)
        ctk.CTkButton(self, text="CANCELAR", fg_color="gray",
                      command=lambda: self.mostrar_menu_post_login(usuario)).pack()

    def limpiar_interfaz(self):
        for widget in self.winfo_children():
            widget.destroy()

if __name__ == "__main__":
    AppCliente().mainloop()

```